



APPLICATION DEVELOPMENT PRACTICES

Dr.Lê Hùng Tiến
VLU



APPLICATION DEVELOPMENT PRACTICES

Using Your Configuration Management Processes Part 1



Director's Overview

- Java Development Tools:
 - eclipse
 - svn
 - Ant
 - JUnit
 - CruiseControl (not used in this course)



How the Tools Work Together

- eclipse is an interactive development environment (IDE) used by programmers to edit and debug source code, initiate build and test processes and display results
- svn is a software configuration management system that stores (keeps revisions of) your project's source files as they are updated by programmers using the eclipse (or another) IDE



How the Tools Work Together

- Ant is a program that automatically builds your Java project from source files in your svn repository and creates the project distribution packages



eclipse

- eclipse is the most popular open source development environment for the Java programming language
 - www.eclipse.org
- Use eclipse to write, compile and debug your source code
- Prerequisite: You must have a current version of the Java Development Kit (JDK) installed on your PC or laptop
 - java.sun.com/javase/downloads/index.jsp

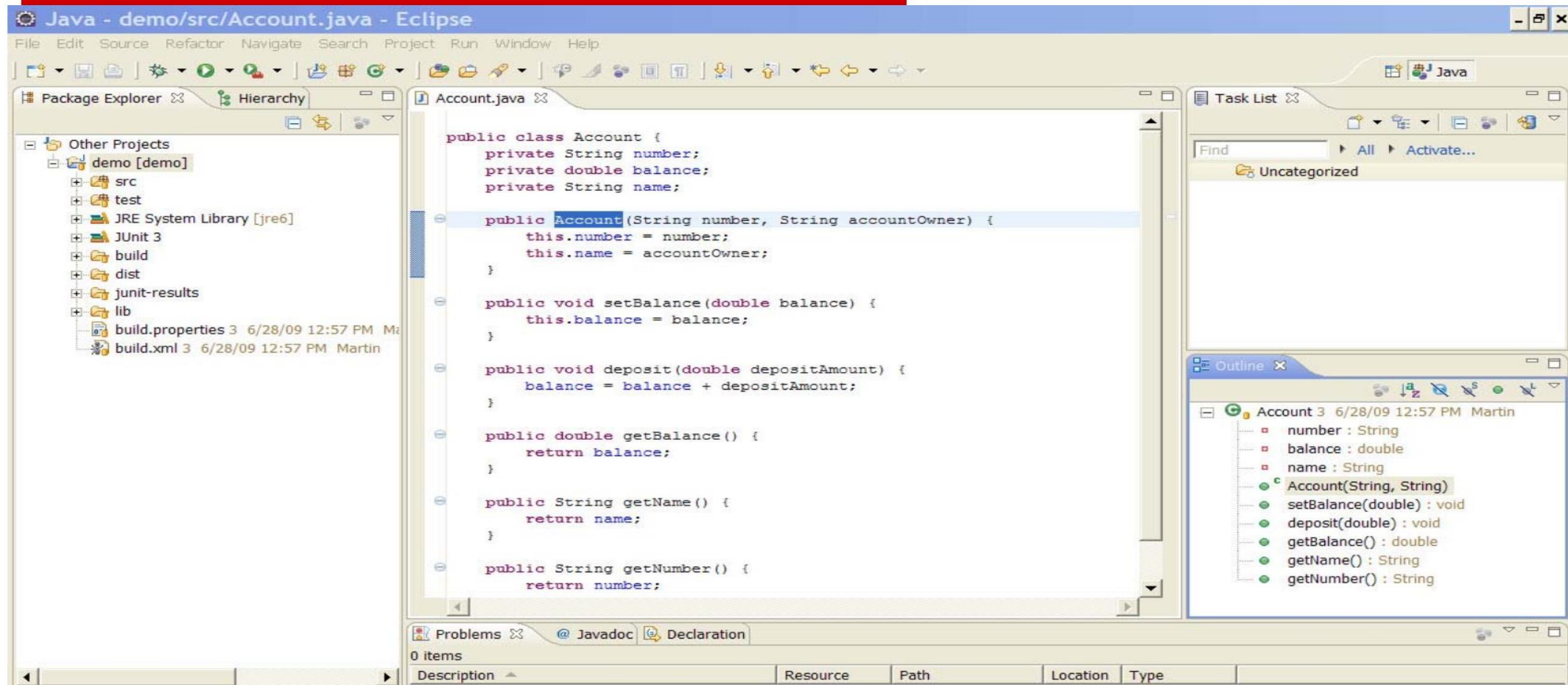


Simple Demo Program

- Tells us how much money is in our account
 - 1 class (account)
 - 3 Fields
 - Account Number
 - Balance
 - Name
 - 4 Methods
 - Get Account Number
 - Set Balance
 - Get Balance
 - Get Name
 - 1 test case



eclipse – Demo Project





eclipse – Account Class

```
Account.java

public class Account {
    private String number;
    private double balance;
    private String name;

    public Account(String number, String accountOwner) {
        this.number = number;
        this.name = accountOwner;
    }

    public void setBalance(double balance) {
        this.balance = balance;
    }

    public void deposit(double depositAmount) {
        balance = balance + depositAmount;
    }

    public double getBalance() {
        return balance;
    }

    public String getName() {
        return name;
    }

    public String getNumber() {
        return number;
    }
}
```



svn

- Subversion is used to maintain current and historical versions of source code, web pages, documentation and other files
 - subversion.tigris.org
- svn integrates with eclipse through a plug-in called "subclipse"

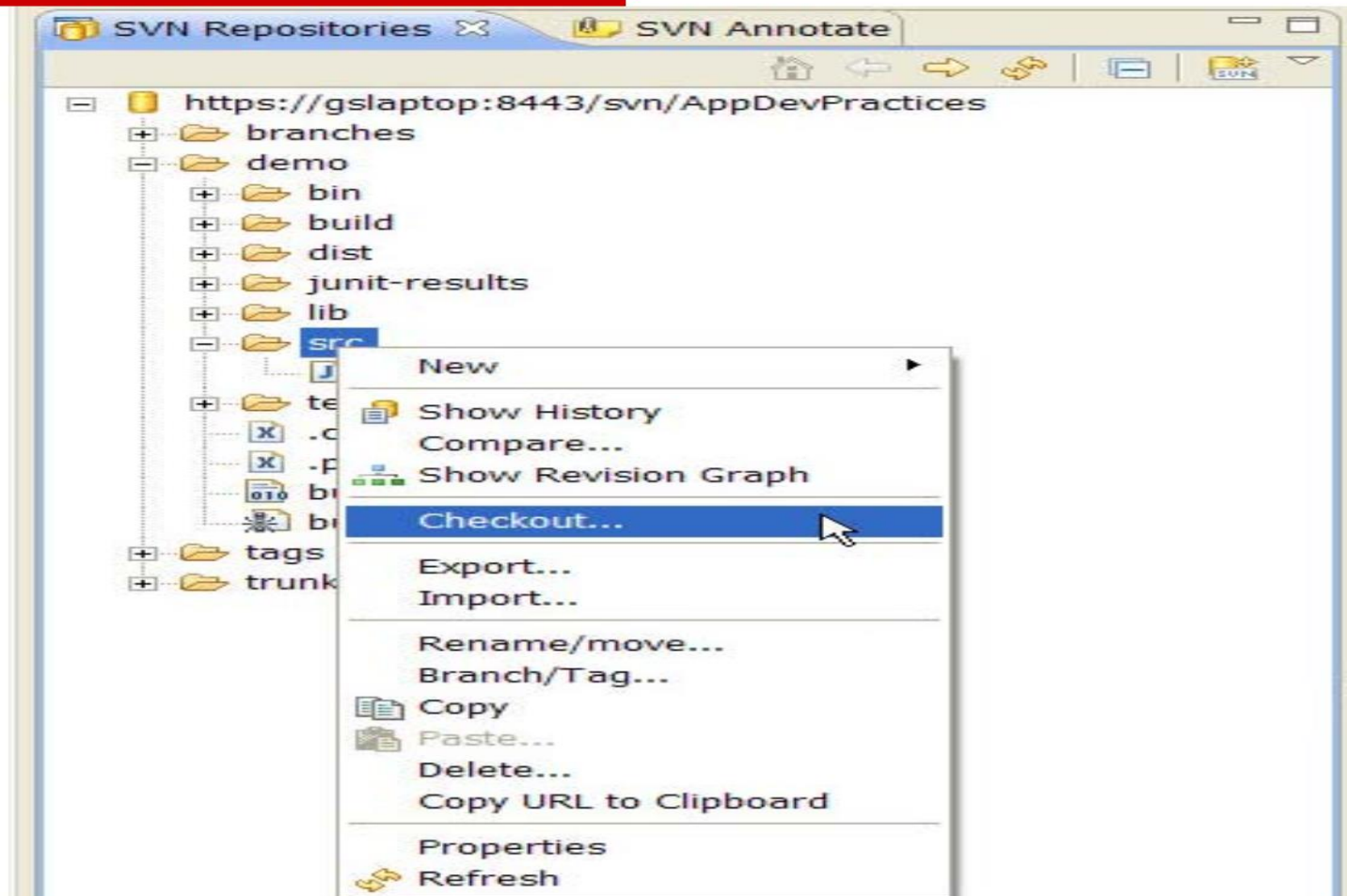


svn Working Copy

- Your working copy is your own private work area containing a collection of files
 - You can edit these files however you wish, and if they're source code files, you can compile them
 - You can have multiple working copies of the same project
- Populate your working copy with
 - **checkout**: will populate your working copy with latest code in repository
 - **update**: will populate your working copy with latest code in repository, but won't overwrite your local changes



eclipse – svn checkout





svn Working Cycle

- Checkout of files from repository into working copy
- Local edit of file
- Local unit and integration testing
- Update (to get changed files from repository)
- Local unit and integration testing
- Commit file back to repository
 - Commit will update the repository with your changes



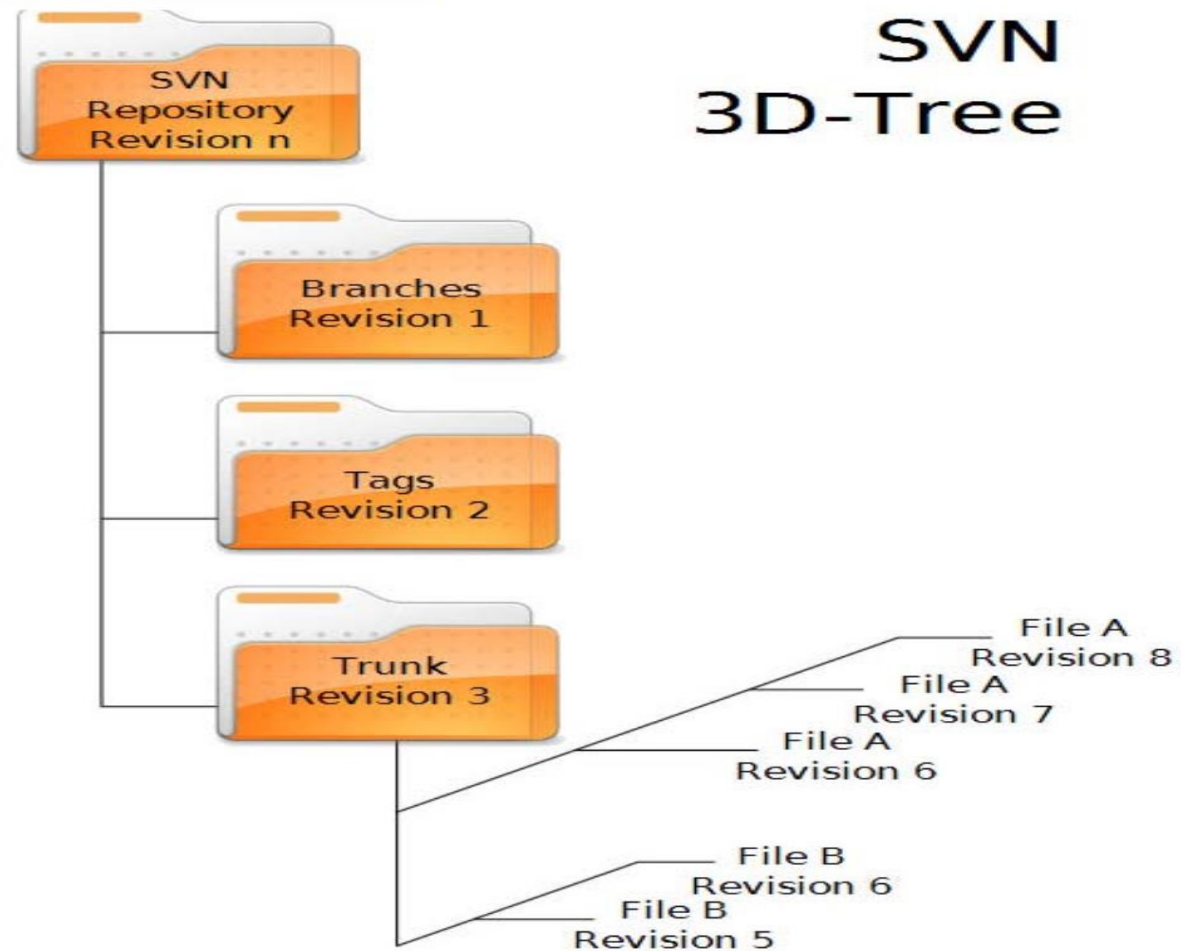
eclipse – svn update / commit

New		
Open	F3	
Open With		
Open Type Hierarchy	F4	
Open Call Hierarchy	Ctrl+Alt+H	
Show In	Alt+Shift+W	
Copy	Ctrl+C	
Copy Qualified Name		
Paste	Ctrl+V	
Delete	Delete	
Remove from Context	Ctrl+Alt+Shift+Down	
Build Path		
Source	Alt+Shift+S	
Refactor	Alt+Shift+T	
Import...		
Export...		
References		
Declarations		
Refresh	F5	
Assign Working Sets...		
Toggle Class Load Breakpoint		
Run As		
Debug As		
Profile As		
Validate		
Team		
Compare With		

Synchronize with Repository
Commit...
Update
Create Patch...
Apply Patch...
Branch/Tag...
Merge...
Switch to another Branch/Tag/Revision...
Show History
Show Annotation...
Add to Version Control
Add to svn:ignore...
Show Properties
Set Property...
Revert...
Edit conflicts
Mark Resolved...
Show Revision Graph
Lock
Unlock
Show Local History
Copy...
Export...



svn Tree Structure





Building Your Java Product

- Simple Java products
 - Simply compile source files
- Complex Java-based projects
 - Retrieve up-to-date source files from source control
 - Manage dependencies not automatically handled by the Java compiler
 - Clean target folders
 - Bundle classes and deliver to appropriate locations (as JAR/WAR files)
- Do you really want to do this manually?

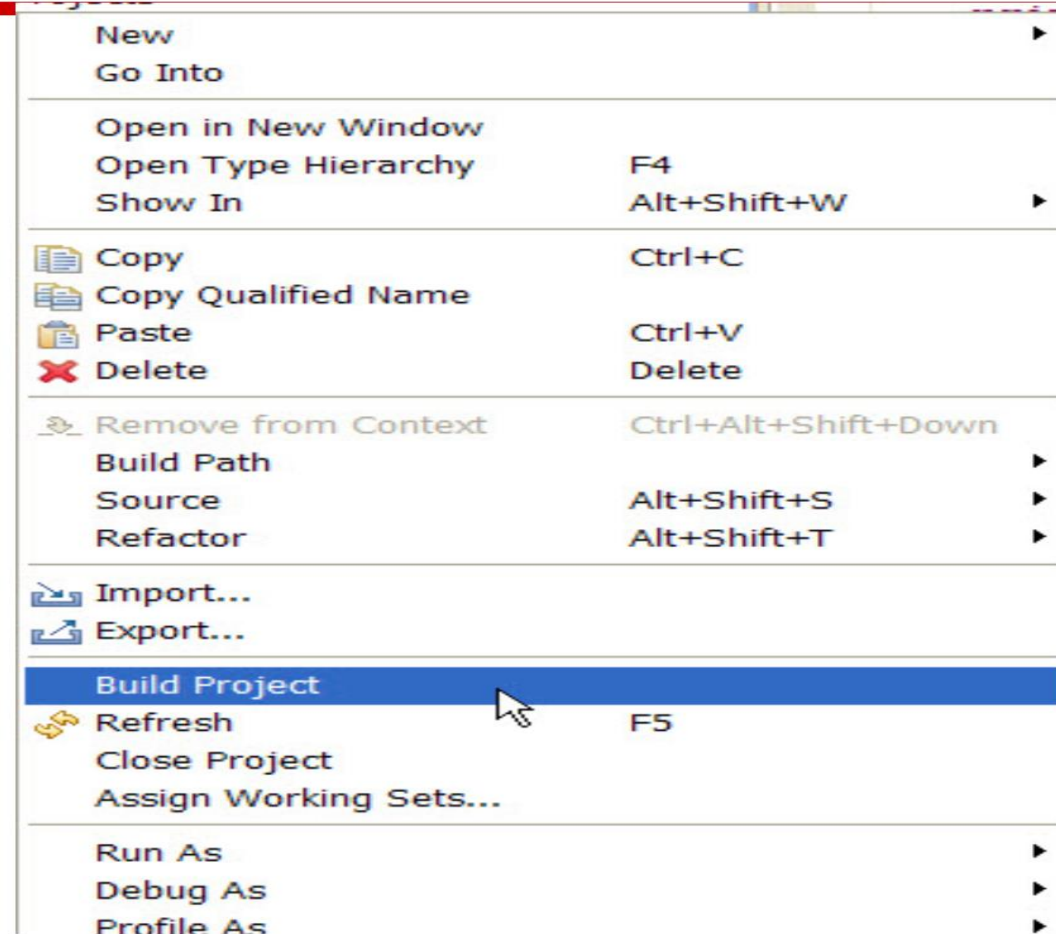


Ant

- Ant is an open source build tool that is the standard for building, packaging and installing Java applications
- Ant uses XML based build files (which makes them much easier to read and modify than “make” files)
- Ant integrates with most Java IDEs
 - Is part of the standard eclipse download
 - ant.apache.org/



Ant Inside eclipse





Build Your Java Product with Make

- Make used to be the standard, but ...
 - Makefiles have their own language syntax, requiring highly trained authors
 - Make lacks platform-independence, requiring multiple versions of the same makefile (one for each target platform) to be maintained and distributed
- The time and maintenance required to use Make is too high for complex Java-based projects



Summary - 1

- Build, compile and debug software
 - eclipse
- Build, compile and debug software where source code is stored in a shared repository
 - eclipse
 - svn
- Build, compile and debug software where source code is stored in a shared repository and builds are automated
 - eclipse
 - svn
 - Ant



Summary - 2

- Different tools for different environments and development languages
 - eclipse (NetBeans, ...)
 - svn (CVS, ...)
 - Ant (Make, NAnt, Maven, Phing, Rake, ...)
- Take advantage of the many good open source tools to increase:
 - Your productivity
 - The quality of your code